

Exercise: aquaculture

Martin Johnsson

Rainbow trout breeding structure (Kurta et al. 2023)

In aquaculture breeding, the “GENERIC” population history of AlphaSimR is a poor fit. Farmed fish were domesticated very recently compared to terrestrial livestock. Here, we use a historical population with the effective size of 1000, and sample founders from it – finding good parameters for rainbow trout is an open question.

```
library(AlphaSimR)
```

Loading required package: R6

```
founders <- runMacs2(nInd = 300,  
                    nChr = 5,  
                    Ne = 1000,  
                    histNe = NULL,  
                    histGen = NULL)
```

We set up two correlated traits, representing measurements at two ages.

```
simparam <- SimParam$new(founders)  
simparam$setSexes("yes_sys")  
  
effect_correlation_matrix <- matrix(  
  c(1, 0.23,  
    0.23, 1),  
  ncol = 2,  
  byrow = TRUE  
)  
  
simparam$addTraitA(nQtlPerChr = 100,
```

```

      mean = c(160, 500),
      var = c(15^2, 30),
      corA = effect_correlation_matrix)

simparam$setVarE(h2 = c(0.39, 0.23))

```

Bookkeeping. We create a data structure to store the populations, and vectors of numbers of sires, dams, viable crosses and offspring.

```

founderpop <- newPop(founders, simParam = simparam)

n_sires <- c(67, 58)
n_dams <- c(133, 116)
n_viable_crosses <- c(133, 73)

n_offspring <- c(1768, 1643)

generations <- vector(mode = "list", length = 2)

```

The cross plan

This function implements a cross plan similar to what is described in the paper. The result is unlikely to be an exact fit; it aims to produce a breeding structure with the number of offspring and families given in the paper.

The function returns a cross plan in matrix form that can be fed to `makeCross2`.

```

make_cross_plan <- function(sires, dams, n_viable_crosses, n_offspring) {
  n_dams <- dams@nInd
  n_sires <- sires@nInd
  offspring_per_family <- n_offspring/n_viable_crosses

  families <- matrix("", nrow = n_dams * 2, ncol = 2)

  ## Create families where each dam is used twice, and sires are used
  ## at random, but at least once
  families[, 1] <- rep(dams@id, each = 2)
  families[1:n_sires, 2] <- sample(sires@id)
  n_extra_sires <- nrow(families) - n_sires
  families[(n_sires + 1):nrow(families), 2] <-
    sample(sires@id, n_extra_sires, replace = TRUE)

```

```

## Sample successful crossings
successful_families <- families[sample(1:nrow(families), n_viable_crosses),]

## Calculate offspring per family; first make equal family sizes,
## then randomly apportion the extra offspring
n_per_family <- rep(floor(offspring_per_family), each = nrow(successful_families))
n_extra_offspring <- n_offspring - sum(n_per_family)
offspring_to_add <- sample(1:length(n_per_family), n_extra_offspring)
for (extra in 1:length(offspring_to_add)) {
  n_per_family[offspring_to_add[extra]] <- n_per_family[offspring_to_add[extra]] + 1
}

## Expand the mating plan by replicating each row the right number of times
cross_plan_list <- vector(mode = "list", length = nrow(successful_families))
for (family in 1:length(cross_plan_list)) {
  cross_plan_list[[family]] <- matrix(rep(successful_families[family, ],
                                          n_per_family[family]),
                                     ncol = 2, byrow = TRUE)
}
cross_plan <- Reduce(rbind, cross_plan_list)
cross_plan
}

```

Generation G0: no selection

The founder generation G0 is created by randomly selecting the appropriate number of founders.

```

sires <- selectInd(founderpop,
  sex = "M",
  method = "rand",
  nInd = n_sires[1],
  simParam = simparam)

dams <- selectInd(founderpop,
  sex = "F",
  method = "rand",
  nInd = n_dams[1],
  simParam = simparam)

cross_plan <- make_cross_plan(sires, dams, n_viable_crosses[1], n_offspring[1])

```

```
generations[[1]] <- makeCross2(females = dams,
                              males = sires,
                              crossPlan = cross_plan,
                              simParam = simparam)
```

G1: Family-based selective breeding

For the G1 generation, we perform family-based selective breeding, selecting first the top families and then the top individuals from each family.

We perform phenotypic selection on an index with equal weight on both traits.

```
candidate_sire_families <- selectFam(generations[[1]],
                                     sex = "M",
                                     trait = selIndex,
                                     b = c(1, 1),
                                     nFam = n_sires[2],
                                     simParam = simparam)

sires <- selectWithinFam(candidate_sire_families,
                        trait = selIndex,
                        b = c(1, 1),
                        nInd = 1,
                        simParam = simparam)

candidate_dam_families <- selectFam(generations[[1]],
                                     sex = "F",
                                     trait = selIndex,
                                     b = c(1, 1),
                                     nFam = n_dams[2],
                                     simParam = simparam)

dams <- selectWithinFam(candidate_dam_families,
                       trait = selIndex,
                       b = c(1, 1),
                       nInd = 1,
                       simParam = simparam)

cross_plan <- make_cross_plan(sires, dams, n_viable_crosses[2], n_offspring[2])

generations[[2]] <- makeCross2(females = dams,
```

```
males = sires,  
crossPlan = cross_plan,  
simParam = simparam)
```

Exercises

Choose the exercises that interest you.

- **More generations.** Write a breeding loop that performs selective breeding for a given number of generations (e.g., 20). To do this you need to wrap the G1 selection code in a loop or a function, and also figure out a model for the number of viable crosses for the coming generations. We suggest keeping the numbers of dams and sires constant at some sensible values and draw the number of viable families from a probability distribution. Save the results and plot genetic means over generations.
- **Family number and size.** One might predict that the number and size of families should affect the genetic gain. Change the number of families in future generations while keeping the total number of offspring per generation constant. Plot the effect on genetic gain. You should probably also replicate the simulation to make sure that any differences you see are not due to random variation.
- **Population history.** Simulate different founder populations with variable effective population size (e.g., 100, 500, 1000) and look at the resulting allele frequency distributions. What happens when you change the historical effective population size? Also run using the “GENERIC” population history that we’ve used before for terrestrial livestock and compare. To speed things up, simulating only one chromosome is fine.
- **Genomic evaluation.** Use the `RRBLUP` function to implement genomic prediction so that you can make selections within family based on GEBVs instead of phenotype. In order to do this you need to: 1) add a SNP chip `simparam$addSnpChip()`, 2) set up a training population made out of older animals with records, 3) run GBLUP with the `RRBLUP` function, 4) use the `setEBV` function to set EBVs of the selection candidates, and finally 5) add the `use="ebv"` argument. Because genomic evaluation takes some time to run, you might want to keep the training population small and only run a few time steps with genomic evaluation.